```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano 19.sh
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ bash 19.sh
19.sh: line 10: syntax error near unexpected token `('
19.sh: line 10: `int change_directory(const char *path, char *previous_dir)'
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <sys/stat.h>

#define MAX_INPUT 1024

int change_directory(const char *path, char *previous_dir)
{
    char current_dir[PATH_MAX];

    if (getcwd(current_dir, sizeof(current_dir)) == NULL)
    {
        perror("getcwd");
        return 0;
    }

    if (chdir(path) != 0)
    {
        perror("cd");
        return 0;
    }

    strcpy(previous_dir, current_dir);

    return 1;
}

void print_current_directory()
{
    char current_dir[PATH_MAX];

    if (getcwd(current_dir, sizeof(current_dir)) == NULL)
    {
        perror("getcwd");
        return;
    }

    printf("%s\n", current_dir);
}

int main()
```

```
            {
                if (strlen(previous_dir) == 0)
                {
                    printf("Previous directory not available.\n");
                    continue;
                }

                char temp[PATH_MAX];

                if (getcwd(temp, sizeof(temp)) == NULL)
                {
                    perror("getcwd");
                    continue;
                }

                if (chdir(previous_dir) == 0)
                {
                    strcpy(previous_dir, temp);
                    print_current_directory();
                }
                else
                {
                    perror("cd");
                }
            }
            else
            {
                if (change_directory(path, previous_dir))
                {
                    printf("Current directory: ");
                    print_current_directory();
                }
            }
        }
        else if (strcmp(command, "exit") == 0)
        {
            break;
        }
        else
        {
            printf("Invalid command: %s\n", command);
        }
    }

    return 0;
}
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m20.sh
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ bash m20.sh
m20.sh: line 9: syntax error near unexpected token `('
m20.sh: line 9: `typedef void (*builtin_function)(char **args);'
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```c
#define MAX_INPUT 1024
#define MAX_ARGS 64

typedef void (*builtin_function)(char **args);

void builtin_cd(char **args)
{
    if (args[1] == NULL)
    {
        printf("cd: missing directory\n");
        return;
    }

    if (chdir(args[1]) != 0)
    {
        perror("cd");
        return;
    }

    printf("Directory changed successfully.\n");
}

void builtin_pwd(char **args)
{
    char current_dir[1024];

    if (getcwd(current_dir, sizeof(current_dir)) != NULL)
        printf("%s\n", current_dir);
    else
        perror("pwd");
}

void builtin_echo(char **args)
{
    int i = 1;

    while (args[i] != NULL)
    {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");

        i++;
    }

    printf("\n");
```

```
    printf("\n");
}

void builtin_help(char **args)
{
    printf("\nBuilt-in commands:\n");
    printf("  cd <directory>   Change directory\n");
    printf("  pwd              Print working directory\n");
    printf("  echo <text>      Print text\n");
    printf("  help             Display help\n");
    printf("  exit             Exit program\n");
}

void builtin_exit(char **args)
{
    exit(0);
}

struct builtin
{
    const char *name;
    builtin_function function;
};

struct builtin dispatch_table[] =
{
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"echo", builtin_echo},
    {"help", builtin_help},
    {"exit", builtin_exit}
};

int builtin_count =
    sizeof(dispatch_table) /
    sizeof(dispatch_table[0]);

int find_builtin(const char *command)
{
    int i;

    for (i = 0; i < builtin_count; i++)
    {
        if (strcmp(command,
                   dispatch_table[i].name) == 0)
        {
```

```
int builtin_count =
    sizeof(dispatch_table) /
    sizeof(dispatch_table[0]);

int find_builtin(const char *command)
{
    int i;

    for (i = 0; i < builtin_count; i++)
    {
        if (strcmp(command,
                   dispatch_table[i].name) == 0)
        {
            return i;
        }
    }

    return -1;
}

int parse_command(char *input, char **args)
{
    int count = 0;

    char *token =
        strtok(input, " \t\n");

    while (token != NULL &&
           count < MAX_ARGS - 1)
    {
        args[count++] = token;

        token =
            strtok(NULL, " \t\n");
    }

    args[count] = NULL;

    return count;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
```

```
    char *args[MAX_ARGS];

    printf("============================================\n");
    printf("       BUILT-IN COMMAND DISPATCH TABLE\n");
    printf("============================================\n");

    printf("Type 'help' for commands.\n");

    while (1)
    {
        printf("\nshell> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL)
        {
            break;
        }

        if (input[0] == '\n')
            continue;

        parse_command(input, args);

        if (args[0] == NULL)
            continue;

        int index =
            find_builtin(args[0]);

        if (index == -1)
        {
            printf("Invalid command: %s\n",
                   args[0]);

            printf("Use 'help' to see valid commands.\n");

            continue;
        }

        dispatch_table[index].function(args);
    }

    return 0;
}
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```